Python Programs Comparing With and Without NumPy

B.HEMA NAGAMANI, Assistant Professor in DS,AUS

1. Creating an Array

Without NumPy:

```
arr = [10, 20, 30, 40, 50]
print(arr)
```

With NumPy:

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr)
```

2. Adding Two Arrays

Without NumPy:

```
a = [1, 2, 3]
b = [4, 5, 6]
result = []
for i in range(len(a)):
    result.append(a[i] + b[i])
print(result)
```

With NumPy:

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b)
```

3. Multiplying Two Arrays

Without NumPy:

```
a = [2, 3, 4]
b = [5, 6, 7]
```

D.L.P.SRI RAM - DAE

```
result = []
for i in range(len(a)):
    result.append(a[i] * b[i])
print(result)
```

With NumPy:

```
import numpy as np
a = np.array([2, 3, 4])
b = np.array([5, 6, 7])
print(a * b)
```

4. Finding Sum of Array Elements

Without NumPy:

```
arr = [10, 20, 30, 40]
total = 0
for i in arr:
    total += i
print(total)
```

With NumPy:

```
import numpy as np
arr = np.array([10, 20, 30, 40])
print(np.sum(arr))
```

5. Finding Maximum Value

Without NumPy: Use a loop to compare elements.

With NumPy:

```
import numpy as np
arr = np.array([12, 45, 7, 89, 23])
print(np.max(arr))
```

6. Finding Average

Without NumPy: Add all values and divide by length.

With NumPy:

```
import numpy as np
```

Dr.P.SRI RAM - DAE

```
arr = np.array([10, 20, 30, 40, 50])
print(np.mean(arr))
```

7. Matrix Addition

Without NumPy: Use nested loops to add matrix elements.
With NumPy:

```
import numpy as np
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
print(A + B)
```

8. Matrix Multiplication

Without NumPy: Use three nested loops.
With NumPy:

```
import numpy as np
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
print(np.dot(A, B))
```

9. Finding Even Numbers

Without NumPy: Use a loop and condition.
With NumPy:

```
import numpy as np
arr = np.array([1,2,3,4,5,6])
print(arr[arr % 2 == 0])
```

10. Reshaping an Array

Without NumPy: Use list slicing.
With NumPy:

```
import numpy as np
arr = np.array([1,2,3,4,5,6])
print(arr.reshape(2,3))
```

DR.P.SRIRAM - DAE

Main Difference Between Python Lists and NumPy Arrays

Without NumPy: Uses Python lists, requires more manual loops, and is slower for large data.
With NumPy: Uses optimized arrays, provides built-in mathematical operations, and is faster for scientific computing.

Heading of the Program	Advantage of using NumPy (Time Complexity, Space Complexity & Other Comparisons)
1. Creating an Array	**Time Complexity:** O(n) for both Python lists and NumPy arrays.**Space Complexity:** O(n) for both.**Advantages:** NumPy stores elements in contiguous memory, resulting in better cache utilization, lower memory overhead, faster access, and support for vectorized numerical operations.
2. Adding Two Arrays	**Time Complexity:** O(n) for both approaches.**Space Complexity:** O(n) for the output array.**Advantages:** NumPy performs element-wise addition using optimized C routines instead of Python loops, reducing execution time and improving code readability.
3. Multiplying Two Arrays	**Time Complexity:** O(n) for both approaches.**Space Complexity:** O(n).**Advantages:** NumPy executes element-wise multiplication using vectorized operations, providing much faster performance and simpler code than manual looping.
4. Finding Sum of Array Elements	**Time Complexity:** O(n) for both approaches.**Space Complexity:** O(1).**Advantages:** np.sum() is implemented in optimized compiled code, minimizing Python interpreter overhead and providing faster execution on large datasets.
5. Finding Maximum Value	**Time Complexity:** O(n) for both approaches.**Space Complexity:** O(1).**Advantages:** np.max() scans the array efficiently using optimized native code, resulting in better performance, concise syntax, and improved maintainability.
6. Finding Average	**Time Complexity:** O(n) for both approaches.**Space Complexity:** O(1).**Advantages:** np.mean() computes the average using optimized numerical routines with better precision, less code, and faster execution than manual computation.
7. Matrix Addition	**Time Complexity:** O(m × n) for both approaches, where *m* and *n* are the matrix dimensions.**Space Complexity:** O(m × n).**Advantages:** NumPy performs matrix addition using vectorized operations without nested loops, improving readability and significantly increasing performance for larger matrices.
8. Matrix Multiplication	**Time Complexity:** $O(n^3)$ using the standard multiplication algorithm.**Space Complexity:** $O(n^2)$.**Advantages:** np.dot() leverages highly optimized

DR PSN RAM DAE

	BLAS/LAPACK libraries, providing substantial speed improvements, better CPU utilization, and efficient handling of large matrices.
9. Finding Even Numbers	**Time Complexity:** O(n) for both approaches.**Space Complexity:** O(k), where k is the number of even elements.**Advantages:** NumPy uses Boolean indexing for filtering, eliminating explicit loops and producing shorter, faster, and more maintainable code.
10. Reshaping an Array	**Time Complexity:** O(1) when a view is returned; O(n) if a copy is required.**Space Complexity:** O(1) for a view and O(n) for a copied array.**Advantages:** reshape() changes the array dimensions without manual indexing or slicing, often without allocating additional memory, making it both efficient and easy to use.

Dr.P.SRI RAM - DAE